

Improvement Backlog, Grounded in Recent Research

Reconstruction-based multivariate time-series anomaly detection. Baseline: validation F1 $\approx$ 0.60, portal F1 $\approx$ 0.63, ROC-AUC $\approx$ 0.76. Literature window 2024–2026.

Recommendations are derived from paper abstracts and method summaries and mapped to this project's code. 'Payoff' is a hypothesis to test, not a promise — benchmarks (TSB-AD, TAB) show no method wins universally.

1. Three framing facts from the literature

- **Reconstruction-based (your family) is still competitive**, but effectiveness depends on anomaly type; your anomalies are slow drifts. A fancier model is not guaranteed to help. [Surveys 2024–25]
- **Contaminated training data is a first-class research problem in 2024–26** — exactly your #1 bottleneck ('train is not anomaly-free'). This is where the literature helps you most. [CLEANet, RiAD, TSAD-C, Encode-then-Decompose]
- **Much published F1 (0.9+) is inflated by point-adjustment**; one study showed random noise can beat SOTA under it. Your un-adjusted 0.63 is not comparable to those numbers — and it means you should verify your portal's exact metric. [Towards Unbiased Evaluation 2024; PATE 2024]

2. Backlog A — Contamination-robust training (highest leverage)

Your teammate's robust_downweight in train.py is a crude version of this whole family. The papers give principled upgrades.

#	Improvement	Source	Maps to your code	Effort	Payoff
A 1	kNN local-density loss weighting. Weight each window's reconstruction loss by a contamination score = similarity to its k nearest neighbours; sigmoid-down-weight outliers.	CLEANet (AWRL), 2025	Replace the top-percentile rule in train.py with a kNN-density weight	Med	High
A 2	Dynamic (iterative) re-weighting. Recompute sample weights across epochs so persistently high-loss (likely-anomalous) windows fade out of training.	RiAD, 2024	Wrap the training loop in train.py with an epoch-wise weight update	Med	High
A 3	Masking-based decontamination. Randomly mask parts of each input and reconstruct them; because normals dominate, masking dilutes anomaly influence.	TSAD-C, 2024 (+6.3% F1)	New masking augmentation in SensorDataset / train step	Med	Med-High
A 4	Encode-then-decompose. Split the latent into a stable (normal) and auxiliary (anomaly-absorbing) part; score with mutual information, not raw error.	Encode-then-Decompose, 2025	Architectural change to ConvAutoencoder1D + new score	High	Med

3. Backlog B — Architecture & representation

#	Improvement	Source	Maps to your code	Effort	Payoff
B 1	Multi-scale convolutions. Add parallel branches mixing random + dilated convs to capture features at several temporal scales.	Hybrid conv-AE for robotics, 2024	Extend DilatedResidualBlock stack in model.py	Med	Med

B 2	Two-stream temporal + feature encoder. One encoder over time, one over the transposed input (cross-sensor correlations). Lightweight (CLEANet: -90% params vs transformer).	CLEANet, 2025	Second encoder branch in ConvAutoencoder1D	Med	Med
B 3	Association-discrepancy scoring. Score by how a point's learned attention/association pattern differs from normal — complements reconstruction error.	Anomaly-Transformer / VAE+AssocDisc, 2025	New scoring head; larger change than Mahalanobis	High	Uncertain
B 4	Explicit inter-sensor graph. Learn a dynamic adjacency (self-attention) over the 18 sensors to model their relationships (time-then-graph).	TSAD-C, 2024	Graph module before/after the encoder	High	Uncertain
B 5	Variational (VAE) bottleneck. Probabilistic latent; surveys report VAEs do well on pattern-wise anomalies.	Surveys 2024–25; math13071209	Swap bottleneck + add KL term in train.py	High	Uncertain

4. Backlog C — Scoring (cheap, aligned with what you already do)

#	Improvement	Source / rationale	Maps to your code	Effort	Payoff
C 1	Fit the Mahalanobis normal model on TRAIN (abundant) rather than the few val normal windows — better-conditioned covariance. Fit on lowest-error windows to dodge contamination.	Covariance estimation best-practice	fit_mahalanobis_from_validation → train	Low	Med
C 2	Combine two scores (e.g. masked-reconstruction + full-reconstruction, weighted).	TSAD-C $s = \lambda_1 \cdot s_1 + \lambda_2 \cdot s_2$	Extend scoring in run_cnn.py	Low-Med	Med
C 3	Seed-ensemble scores. Average per-timestep scores over 3–5 CNNs (different seeds) — variance reduction typically lifts AUC.	Ensembling best-practice	New run_cnn_ensemble pipeline	Low	Med

5. Backlog D — Evaluation & threshold (protect the F1 you have)

#	Improvement	Source	Maps to your code	Effort	Payoff
D 1	Confirm the portal metric, then match the threshold objective to it. You optimise F0.5; if the portal grades F1, tune the threshold for F1.	your setup + eval papers	beta in find_best_threshold / config	Low	Med (free F1)
D 2	Report F1-BA / PATE internally for honest self-comparison; do not compare your 0.63 to point-adjusted paper numbers.	Unbiased Eval 2024; PATE 2024	Add to metrics.py (reporting only)	Low	Rigor
D 3	Quantile-based threshold as a robust alternative/cross-check to the PR-curve sweep.	TSAD-C thresholding	Option in find_best_threshold	Low	Low-Med

6. Recommended sequence (effort-adjusted)

Do the cheap, aligned, high-confidence items first; treat the big architectural bets as later research, only if the ceiling is still binding.

Order	Item	Why first
-------	------	-----------

0	Diagnostic plots (score-vs-label per run; normal/anomaly histogram)	See the failure mode before changing anything; confirms contamination is visible
1	D1 — confirm portal metric & align threshold	Possibly free F1, minutes of work, zero model risk
2	A1/A2 — kNN-density robust training (upgrade robust_downweight)	Highest-leverage lever; attacks the AUC ceiling at its source; code partly exists
3	C3 — seed-ensemble + C1 — Mahalanobis-on-train	Cheap, low-risk AUC lifts; also stabilises your noisy single-run F1
4	A3 — masking decontamination	Well-evidenced (+6.3% F1 in TSAD-C); moderate effort
5	B1/B2 — multi-scale / two-stream encoder	Architectural gains after the cheaper wins are exhausted
6	A4/B3/B4/B5 — decompose / attention / graph / VAE	Big research bets; high effort, uncertain payoff; last

Every experiment must change ONE thing and be measured with run-grouped CV or seed averaging — a single run varies by ± 0.02 F1, so smaller 'gains' prove nothing.